

FlexSpec: 在边缘 - 云协作 LLM 推测解码中冻结草稿与不断变化的目标相遇

李宇晨, 孔瑞, 吕中浩, 李启洋, 陈欣然, 蔡恒毅, 闫凌勇, 王帅强, 赵嘉树, 朱光旭, 孔令和, IEEE 会员, 陈贵海, IEEE 会员, 熊浩逸, 尹大伟

摘要 —— 在移动和边缘计算环境中部署大型语言模型

(LLM) 受到设备资源有限、无线带宽稀缺以及模型频繁更新的制约。虽然结合推测解码 (SD) 的边缘 - 云协作推理可以通过在边缘执行轻量级草稿模型并用云端目标模型进行验证来降低端到端延迟, 但现有框架在根本上依赖于两个模型之间的紧密耦合。因此, 重复的模型同步会引入过多的通信开销, 增加端到端延迟, 并最终限制 SD 在边缘环境中的可扩展性。为了解决这些限制, 我们提出了 FlexSpec, 一种针对不断演进的边缘 - 云系统而设计的高效通信协作推理框架。FlexSpec 的核心设计是一种共享骨干架构, 它允许单一且静态的边缘草稿模型保持与大量演变中的模型兼容。

索引词 —— 移动计算、边缘 - 云协作、大型语言模型、推测解码、协作推理。

一、引言

LARGE language models (LLMs), exemplified by Chat-GPT [1], LLaMA [2], 和 Gemini [3], 已成为智能代理的重要引擎, 应用范围从移动个人助手到实时翻译设备。这些模型的能力在很大程度上决定了

Y. Li、R. Kong、Q. Li、X. Chen、H. Cai、L. Yan、S. Wang、H. Xiong 和 D. Yin 就职于中国北京百度公司 (电子邮件: {yuchenli1230, monster119120, qiyangli878, fantastique0910, hengyi1995, lingyongy, shqiang.wang}@gmail.com; haoyi.xiong.fr@ieee.org; yindawei@acm.org)。Y. Li、L. Kong 和 G. Chen 就读于中国上海上海交通大学计算机科学学院 (电子邮件: {yuchenli, linghe.kong, chen-gh}@sjtu.edu.cn)。Z. Lyu 就职于瑞典斯德哥尔摩皇家理工学院信息科学与工程系 (电子邮件: lzhon@kth.se)。(通讯作者: Z. Lyu) J. Zhao 就职于 Wilfrid 大学物理与计算机科学系 (电子邮件: jzhao@wlu.ca)。G. Zhu 就职于中国深圳大数据研究院 (电子邮件: gxzhu@sribd.cn)。

用户体验和复杂推理任务的成功执行 [4]–[7]。尽管许多研究已经集中在通过量化和剪枝等技术在边缘设备上压缩这些模型 [8], 但单个设备上的推理仍受限于电池寿命有限、热节流以及内存带宽不足等因素 [9]。

边缘 - 云协同推理通过将计算密集型子任务卸载到强大的云服务器, 同时在边缘保留轻量级处理, 从而扩展了推理能力并提高响应精度 [10], 以解决这一限制。在这一协同模式中, 一个关键挑战是如何在不牺牲准确性的情况下保持实时性能, 尤其是在高度动态的移动网络环境中。用户查询与生成响应之间的延迟可能导致输出令人沮丧地缓慢, 从而影响应用的交互性和可靠性。在自回归生成中, 这一问题更为突出, 因为逐令牌传输会引入显著的通信延迟。此外, 边缘与云之间交换的信息量通常很大。因此, 迫切需要高效的推理加速方法以减少通信开销和决策延迟。

为了减轻自回归生成的延迟, 推测解码

(Speculative Decoding, SD) 已被广泛采用, 作为一种关键的推理加速技术 [12]。在典型的边缘 - 云 SD 范式中, 边缘设备会快速生成一系列推测性的“草稿”令牌, 这些令牌会被传输到功能强大的云服务器进行并行验证。虽然近期基于深度学习的 SD 方法, 如 EAGLE [13] 和 Medusa [14], 已展示出有希望的延迟降低效果, 但它们在边缘 - 云系统中的部署仍面临若干根本性挑战。具体而言, 现有的 SD 框架依赖于草稿模型与目标模型之间的紧密耦合, 而云端的目标模型经常通过参数高效微调 (Parameter-Efficient Fine-Tuning, PEFT) [11], 进行更新, 从而导致所谓的“更新风暴”, 其特点是在带宽受限的无线链路上维持模型一致性时产生过度通信开销。此外, 通过保持边缘

在移动环境中，这削弱了固定步长推测执行的延迟收益。综合来看，频繁的模型演化、草稿模型与目标模型之间的分布变化以及时变的无线延迟，从根本上限制了现有推测解码框架在实际边缘 - 云部署中的适用性。

受这些挑战的激励，我们的目标是在边缘 - 云系统中实现版本无关的推测解码，即通过单一的静态边缘模型加速多样化、不断发展的云目标模型。一旦我们构建了这个稳健的草稿框架，它可以使用单一的草稿模型对多个微调版本进行加速，同时显著降低维护成本。此外，这个解耦框架对未见过的目标分布具有良好的泛化能力，与必须从零重新训练的现有系统不同。因此，部署和通信成本都大幅降低。

具体来说，受到基础模型（FMs）在模态无关处理方面成功的启发，我们提出了 **FlexSpec**，一种用于演化大型语言模型的通信高效的边缘 - 云协作推测解码框架。FlexSpec 的核心思想是从根本上将边缘侧草稿模型的生命周期与云端目标模型的生命周期解耦，从而消除了带宽受限的无线环境中频繁通过空中进行模型同步的需求。此外，FlexSpec 通过将推测解码行为适应于时变的网络情况，明确考虑了无线信道的随机性。通过在统一框架内联合考虑模型演化和无线动态，FlexSpec 实现了边缘系统中可扩展、鲁棒且低延迟的 LLM 推理。本文的主要贡献总结如下：

- 我们提出了 FlexSpec，一种新型的边缘 - 云协作推测解码框架，可以实现对不断发展的大语言模型版本无关推理。通过在边缘草稿模型和云端目标模型之间引入共享的冻结锚点骨干，FlexSpec 从根本上解耦了它们的生命周期，消除了频繁同步草稿模型的需求，有效解决了无线边缘网络中的“更新风暴”问题。
- 我们开发了一种针对边缘环境下推测解码的信道自适应生成策略。通过对推测步长、标记接受率和通信速率之间的相互作用进行建模，我们推导出了一个理论基础的生成长度选择公式，实现了对时变信道条件的动态适应，并最大化有效标记生成吞吐量。
- 我们在大规模 GPU 集群（最多 32 个 NVIDIA A800 GPU）上对各种推理和编码任务（例如 GSM8K 和 HumanEval）进行了广泛的实验。结果表明，FlexSpec 在端到端生成效率上持续提升，并在异构无线网络中提供稳健的性能。特别是，通过自适应地调整投机步幅以应对时变的信道条件，FlexSpec

在通信受限的环境下，其性能超过最先进的投机解码方法。此外，通过将冻结的边缘草稿模型与频繁更新的云端目标分离，FlexSpec 在很大程度上避免了重复的草稿模型同步，从而显著降低了与更新相关的通信开销。

本文的其余部分组织如下。第二部分介绍相关工作。第三部分介绍边缘 - 云推测解码的系统模型和问题定义。第四部分详细说明所提出的 FlexSpec 的网络架构和自适应策略。第五部分展示实验结果以验证 FlexSpec 的性能。第六部分总结本文。

二、相关工作

在本节中，我们将从用于大语言模型加速的蒸馏、边缘 - 云协作推理以及无线感知移动人工智能的角度回顾和讨论相关工作。

A. LLM 加速的 SD

SD 已经成为一种关键范式，以缓解大规模语言模型自回归生成中固有的受内存限制的延迟瓶颈。SD 的核心前提是通过采用“草稿 - 然后 - 验证”策略，用丰富的并行计算能力换取减少的顺序内存访问。

在基础性工作中，Leviathan 等人 [12] 和 Chen 等人 [15] 对推测执行框架进行了形式化，证明了一个小型草稿模型可以生成候选令牌，然后由目标模型并行验证，从而在数学上保证输出与目标模型完全一致。在此基础上，最近的进展集中在提高草稿质量和接受率。Medusa [14] 在冻结的原始模型之上引入了多个解码头，以同时预测多个未来的令牌，消除了对单独草稿模型的需求。EAGLE [13] 和 EAGLE-2 [16] 通过利用逐层特征外推进一步推进了这一点，在草稿层处理输入特征，以生成更高精度的上下文草稿。除了基于模型的方法外，基于非模型的草稿技术，如 n-gram 匹配 [17] 和基于检索的增强 [18] 利用统计模式或外部数据库来 pr

然而，这些现有的 SD 框架是在静态环境的假设下运行的，即草稿模型和目标模型紧密耦合或结构相同。在边缘场景中，云端目标模型通过 PEFT [19] 频繁演化。维持如 EAGLE 或 Medusa 等方法所施加的严格耦合，将需要通过无线链接频繁同步边缘端的草稿模型与云端目标模型，从而产生无法承受的通信开销。FlexSpec 的不同之处在于，它通过在结构上解耦草稿和目标的生命周期，使单个静态边缘模型能够为不断演化的云端目标服务，而无需频繁同步。

B. 边缘 - 云协同推理

边缘 - 云协作推理弥合了移动设备有限资源容量与现代深度神经网络（DNN）巨大计算需求之间的差距 [20]。这一领域通常研究如何在边缘和云之间优化计算划分，以平衡延迟、能耗和隐私。

早期的工作，如神经外科医生 [21] 和松原等人探索的技术 [22]，重点是确定 DNN 中的最佳分割点以最小化端到端延迟。随着大语言模型（LLM）的兴起，已经提出了分布式推理框架（例如，[23]–[25]），以通过互联网在异构设备之间分配 Transformer 模块。研究人员还提出了激进的量化技术 [26]，[27] 和激活压缩方法 [28]，以缓解传输大型中间激活张量所导致的通信瓶颈。

尽管取得了这些进展，标准的分割推理方法在应用于生成任务时往往存在传输延迟高的问题，因为对每个令牌传输高维激活值需要大量通信。此外，它们在生产环境中大多忽略了版本一致性问题，而在这种环境中云模型每天都会更新。FlexSpec 通过传输轻量级的令牌索引而非大型激活值，并引入基于锚点的对齐机制，使边缘 - 云协作对模型版本不匹配具有鲁棒性，从而推进了该领域的发展。

C. 无线感知移动人工智能

在移动计算环境中部署人工智能应用需要严格考虑无线网络动态。与稳定的有线连接不同，具有衰落和噪声的动态无线信道会显著影响实时人工智能应用的体验质量（QoE）。为了解决这些问题，Park 等人 [29] 和 Zhu 等人 [30] 通过优化计算卸载与通信信道质量之间的权衡，为无线边缘智能奠定了基础。在自然语言处理（NLP）的特定背景下，像 DeeBERT [31] 和 SPINN [32] 这样的动态推理机制允许模型根据置信度阈值提前退出以节省能量。最近，DSSD [33] 尝试将推测解码适配到无线环境中，但其依赖固定策略，未能充分利用实时信道状态信息。

FlexSpec 填补了无线网络理论与大型语言模型推理系统之间的空白。与之前将生成长度 K 视为固定超参数的工作不同，FlexSpec 将起草步长建模为一个依赖于信道条件的动态变量。通过对生成过程应用信道感知的自适应，FlexSpec 确保系统即使在移动场景中典型的波动网络条件下，也能保持高吞吐量。

III. 系统模型与问题定义

在本节中，我们首先介绍边缘 - 云协同推理的系统模型。然后，基于使用真实世界数据集和网络跟踪的初步实验，

我们确定了三个阻碍现有投机解码框架在移动环境中实际部署的关键研究挑战：模型同步的高带宽成本、分布偏移引起的性能崩溃，以及对无线网络波动的敏感性。

A. S 系统模型：边缘 - 云协同信息

参考

我们考虑一个由边缘设备 $\mathcal{E}$ 和云服务器 $\mathcal{C}$ 组成的协作推理系统。云端托管一个高容量的目标模型 $\mathcal{M}_t(\cdot; \theta_t)$ （例如 Llama-2-70B），部署在 GPU 集群上（例如 NVIDIA A800），其中 $\theta_t \in \mathbb{R}^{N_t}$ 表示其参数。为了在不断发展的下游任务上保持竞争性能，服务提供商会定期通过 PEFT 或全参数训练在不断变化的任务数据集 $\mathcal{D}_{\text{task}}$ 上更新 $\mathcal{M}_t$ 。因此，云模型会随着时间演化为一个序列 $\{\mathcal{M}_t^{(0)}, \mathcal{M}_t^{(1)} \dots \mathcal{M}_t^{(s)}\}$ 。

在边缘端，客户端托管一个轻量级的草稿模型 $\mathcal{M}_d(\cdot; \theta_d)$ ，带有 $\theta_d \in \mathbb{R}^{N_d}$ 和 $N_d \ll N_t$ ，其大小通常仅相当于 $\mathcal{M}_t$ 的一小部分（例如，约为单个目标模型层的规模）。由于存储、电池和计算能力的严格限制，边缘设备无法经常下载大型模型权重。因此，它在目标模型更新期间保持静态（冻结），即 $\mathcal{M}_d^{(s)} = \mathcal{M}_d^{(0)}, \forall s$ 。

边缘和云通过随机无线链路进行通信。在每一个推测解码步骤中，边缘起草一块令牌 x_{draft} 并将其传输到云端，同时云端使用 $\mathcal{M}_t$ 验证起草的块，并返回验证结果

$x_{\text{veri edo}}$

给定输入提示 X ，系统生成长度为 O 的输出序列 $Y = \{y_1, \dots, y_O\}$ 。在草稿长度为 K 的推测解码下，一步解码的延迟 n 被建模为

$$T_{\text{step}}(K, R_n) = T_{\text{edge}}(K) + T_{\text{up}}(K, R_n) + T_{\text{cloud}}(K) + T_{\text{down}}, \quad (1)$$

其中 $T_{\text{edge}}(K)$ 是边缘计算处理 K 个标记的时间， $T_{\text{up}}(K, R_n)$ 是上行传输延迟， $T_{\text{cloud}}(K)$ 是云端验证时间， T_{down} 是传递验证结果 / 反馈的下行延迟。此外， $T_{\text{up}}(K, R_n)$ 进一步表示为上行传输开销 $B_{\text{up}}(K)$ 与步骤 n 的可实现上行速率 R_n 之间的比值，即 $T_{\text{up}}(K, R_n) = B_{\text{up}}(K)/R_n$ 。

B. 挑战 1：无线网络中的“更新风暴”

第一个主要障碍是模型新鲜度需求与有限的无线带宽之间的冲突。在标准的 SD 范式中，草稿模型 $\mathcal{M}_d$ 需要近似目标模型 $\mathcal{M}_t$ 的分布。因此，每当 $\mathcal{M}_t$ 更新到新版本 $\mathcal{M}_t'$ （例如，为医疗应用进行微调）时，边缘端 $\mathcal{M}_d$ 理论上必须重新训练并同步，以维持高的令牌接受率 $\hat{\gamma}$ 。

我们将这一现象称为“更新风暴”。为了量化其影响，我们对同步一个轻量级所需的传输开销进行了初步分析

表 I 无线网络上同步草稿模型的估计延迟。频繁更新使得标准 SD 不可行。

Network Type	Bandwidth	Synchronization Time (one User)	Scalability (1k Users)
Public WiFi	10 Mbps	≈ 48 min	Collapse
4G LTE	50 Mbps	≈ 9.5 min	High Congestion
5G Mid-Band	300 Mbps	≈ 1.6 min	Moderate Load

通过典型的移动网络下载草稿模型（大约 3.2 GB）。

如表 I 所示，即使下载一个压缩的草稿模型，在 4G 网络下也需要近 10 分钟。对于每天进行模型更新的移动应用来说，这对蜂窝网络基础设施造成了 PB 级的流量，并迅速消耗用户的数据计划和电池。这促使我们设计了一个静态草稿模型，该模型不需要同步。

C. 挑战 2：分布变化与性能崩溃

如果我们为了节省带宽而放弃模型同步（即在 $\mathcal{M}_d$ 保持固定的同时 $\mathcal{M}_t$ 演化），我们将遇到第二个挑战：分布偏移。静态草稿模型的输出分布 P_d 与不断变化的目标模型的分布 P'_t 在微调后会显著差异。

为验证这一点，我们进行了一个初步实验，使用通用的 Llama-2-7B 作为冻结的草稿模型。我们测量了针对 70B 目标模型三个版本的令牌接受率：基础版本、经过数学调优的版本（GSM8K）以及经过代码调优的版本（HumanEval）。

表 II 目标模型演化对固定草稿模型性能的影响。没有对齐时，接受率急剧下降。

Target Model Version	Domain	Acceptance Rate
Llama-2-70B-Base	General	0.72
Llama-2-70B-Math (LoRA)	Mathematics	0.45 ($\downarrow$ 37.5%)
Llama-2-70B-Code (Full)	Programming	0.18 ($\downarrow$ 75.0%)

表 II 的结果显示了严重的“性能崩溃”。当目标模型发展到编码领域时，通用草稿模型的接受率骤降至 0.18。在如此低的接受率下，推测性解码（运行草稿模型并传输令牌）的开销超过了其收益，往往导致延迟比标准自回归解码更高。这就需要像 FlexSpec 的基于锚点的对齐这样的机制来弥合静态表示与不断演变的表示之间的差距。

D. 挑战 3：无线延迟敏感性

第三个挑战在于推测长度与网络条件之间的耦合。大多数现有工作采用固定的推测步长（例如，）。然而，我们的网络追踪分析表明，在移动环境中固定的可能是次优的。在弱信号场景下（例如，电梯或信号噪声比 (SNR) 小于 dB 的地铁）

(1) 中的上行传输项 $T_{up}(K, R_n)$ 主导了端到端延迟。例如，传输五个令牌可能会产生大约 200 毫秒的上行延迟，而相应的验证收益约为 50 毫秒。相反，在强 5G 信道条件下，一个小的固定 K 不能充分利用可用带宽。这种不匹配产生了一个根本性的困境：静态预测策略无法同时满足异构移动性和信道条件下的延迟要求，从而突出了本文所提出的信道感知自适应预测机制的必要性。

E. 问题定义

基于 (1)，我们的目标是设计一个在不断演化的边缘-云系统中仍然有效的边缘-云 SD 框架。具体来说，考虑到随时间更新的一系列云端目标模型 $\{\mathcal{M}_t^{(s)}\}$ ，我们寻求一种模型耦合机制，使得边缘草稿模型 $\mathcal{M}_d$ 能够保持静态（即无需边缘端重新训练或重复下载模型），同时在与每个 $\mathcal{M}_t^{(s)}$ 协作时保持高令牌接受率。

同时，在时变无线条件下，我们旨在寻求一个关于 K 的策略，以及在 $\mathcal{M}_d$ 和 $\mathcal{M}_t$ 之间的模型耦合机制，以最大化有效令牌生成速率（ETGR），即每单位时间内被接受的令牌的期望数量。设 τ 表示在一次长度为 K 的起草与验证轮中接受的令牌数量，则 ETGR 由长度为 K 的投机性起草所产生的期望接受令牌数量 τ （即 $\mathbb{E}[\tau | K]$ ）与公式 (1) 中的每轮延迟之比来表征。

$$\text{ETGR}(K) \triangleq \frac{\mathbb{E}[\tau | K]}{\mathbb{E}[T_{\text{step}}(K, R_n)]}, \quad (2)$$

这相当于最小化每个被接受的令牌的期望延迟，即 $\mathbb{E}[T_{\text{step}}(K)]/\mathbb{E}[\tau | K]$ 。总之，我们的目标是学习一个关于 K 的自适应策略以及一个在 $\mathcal{M}_d$ 和 $\mathcal{M}_t$ 之间的模型耦合机制，以最大化 ETGR。

四、方法论：FLEXSPEC

在本节中，我们详细介绍了 FlexSpec 框架以应对上述挑战。我们的方法由三个集成组件组成：(1) 一个结构解耦架构，使静态边缘草稿模型能够服务于不断发展的云目标；(2) 一个通道感知推测策略，根据实时通道状况优化草稿长度 K ；以及 (3) 一个语义同步协议，以最小化传输开销。

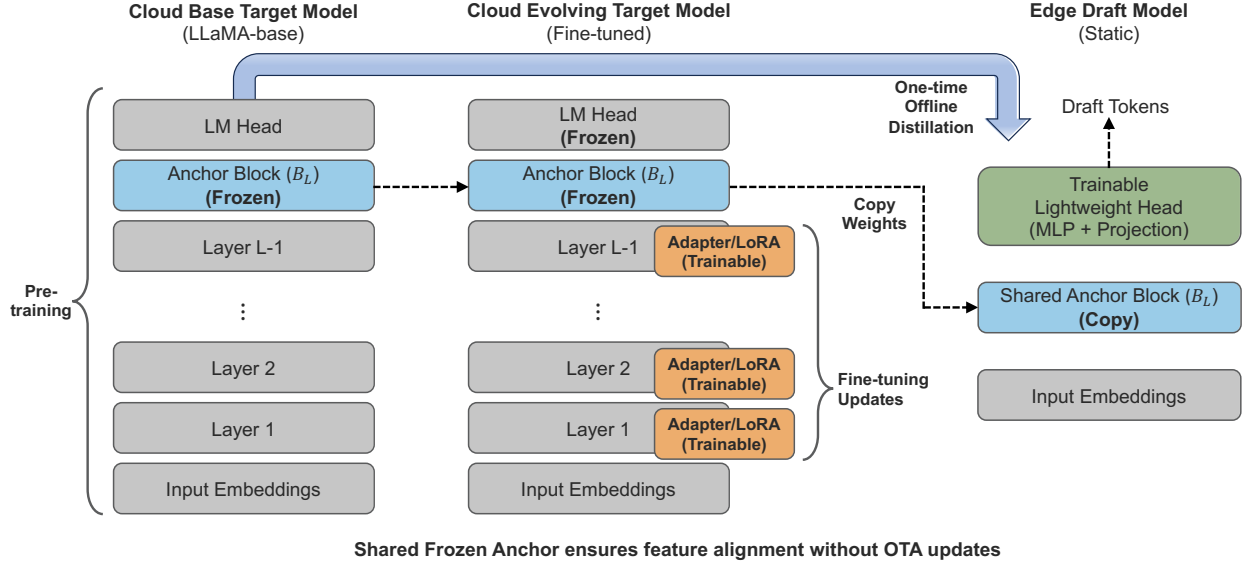


图 1. FlexSpec 训练架构：共享的冻结锚块（ B_L ）确保静态边缘草稿模型与不断发展的云端目标模型之间的特征空间对齐。轻量级头部通过一次性离线蒸馏进行训练。

A. 架构：基于锚点的特征对齐

为了满足 θ_d 保持静态而 θ_t 发生变化的约束，我们提出了基于锚点的对齐架构。当分布 $P_d(y|x)$ 由于云端微调而与 $P_t(y|x)$ 偏离时，标准的推测解码会失败。我们假设，尽管在微调过程中高层语义指令会发生变化，但在骨干网络 [34] 中的基础令牌表示逻辑保持稳定。

1) 结构定义：我们将任务特定微调前的云端目标模型记作 $\mathcal{M}_{\text{base}} \triangleq \mathcal{M}_t^{(0)}$ 。遵循图 1 中的架构，我们将 $\mathcal{M}_{\text{base}}$ 分解为特征提取器和头部，如

$$\mathcal{M}_{\text{base}}(x) = \Psi_{\text{base}}(\Phi_{\text{base}}(x)), \quad (3)$$

其中 Φ_{base} 由底部的变压器堆栈（第 1 层到 $L-1$ 层）以及输入嵌入模块组成， Ψ_{base} 表示包含最后一个变压器块（第 L 层）和语言模型头的头。我们将 Ψ_{base} 内的最后一个变压器块称为锚块，用 $\mathcal{B}_L^{\text{base}}$ 表示。

基于这种分解，边缘草稿模型 $\mathcal{M}_d$ 通过以下方式构建：(i) 从基础头部复制锚块并保持其冻结，(ii) 附加一个轻量可训练的用于标记预测的头部。具体来说，

$$\mathcal{M}_d(x) = \mathcal{H}_{\text{small}}(\mathcal{B}_{\text{shared}}(x)), \quad (4)$$

其中 $\mathcal{B}_{\text{shared}} \triangleq \mathcal{B}_L^{\text{base}}$ 是锚点块的冻结副本， $\mathcal{H}_{\text{small}}$ 是一个轻量可训练的投影头，由一个两层多层感知器（MLP）后跟一个词汇投影矩阵组成。通过这种方式， $\mathcal{M}_d$ 重用基础头所引入的表示空间，同时保持足够紧凑以便于边缘部署。

在云端微调期间，我们采用 PEFT 风格的参数化来对不断演变的的目标模型 $\mathcal{M}_t(\cdot; \theta_t)$ 进行参数化，通过将其参数分解为 $\theta_t = \{\theta_t^{\text{backbone}}, \theta_t^{\text{adapters}}\}$ 。这里， $\theta_t^{\text{backbone}}$ 表示 $\mathcal{M}_{\text{base}}$ 的原始预训练权重（即，

Φ_{base} 和 Ψ_{base} 的参数，包括锚块和语言模型头部，而 $\theta_t^{\text{adapters}}$ 表示注入到变压器层中的额外 PEFT 参数（例如 LoRA/ 适配器模块）。我们通过保持 $\theta_t^{\text{backbone}}$ 冻结并且仅在微调期间更新 $\theta_t^{\text{adapters}}$ 来执行骨干冻结约束。因此，头部 Ψ_{base} （层 L 和语言模型头部）保持不变，这稳定了锚块所看到的特征流形，并保持了静态边缘草稿 $\mathcal{M}_d$ 依赖以实现精确绘制的特征兼容性。

2) 通用训练目标：为了训练静态边缘组件（ $\mathcal{H}_{\text{small}}$ ），我们采用了一个多目标损失函数，结合了特征回归和知识蒸馏（KD）[35]。训练在通用语料库（例如 RedPajama [36]）上进行一次，并使用基础模型 $\mathcal{M}_{\text{base}}$ 作为教师模型。具体的损失函数如下所示。

• **特征回归损失（ $\mathcal{L}_{\text{feat}}$ ）**： $\mathcal{L}_{\text{feat}}$ 确保草稿隐藏状态 h_d 与目标隐藏状态 h_t 对齐，即，

$$\mathcal{L}_{\text{feat}} = \frac{1}{\omega \cdot \pi} \sum_{i=1}^{\omega} \sum_{j=1}^{\pi} \|W_p \cdot h_d^{(i,j)} - h_t^{(i,j)}\|_2^2, \quad (5)$$

其中 W_p 是一个可学习的投影矩阵，用于匹配维度 [37]， ω 是批量大小， π 是序列长度。

• **软目标 KD 损失（ $\mathcal{L}_{\text{KD}}$ ）**：它最小化令牌分布之间的 KL 散度 [38]，即，

$$\mathcal{L}_{\text{KD}} = \mathcal{T}^2 D_{\text{KL}} \left(\sigma \left(\frac{z_t}{\mathcal{T}} \right) \parallel \sigma \left(\frac{z_d}{\mathcal{T}} \right) \right), \quad (6)$$

其中 z_t, z_d 是 logits， σ 是 Softmax 激活函数， $\mathcal{T}$ 是温度。

那么总目标就是（5）和（6）的结合，即

$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{feat}} + \lambda_2 \mathcal{L}_{\text{KD}}$ 。算法 1 总结了一次性不精确训练过程。

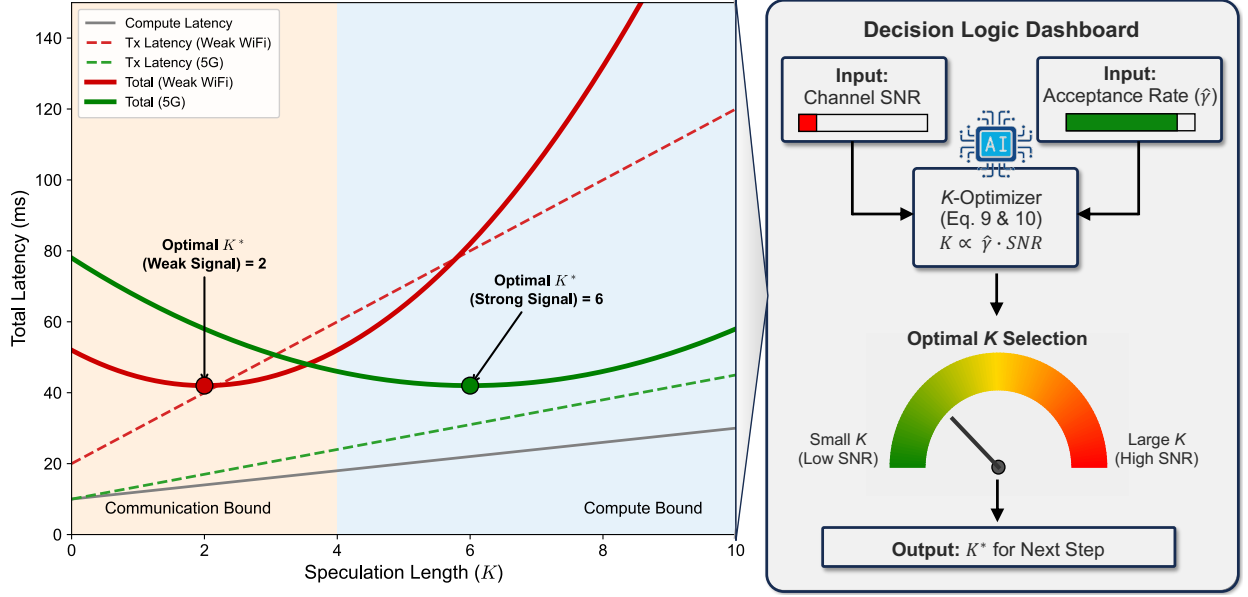


图 2. 通道感知自适应推测机制：左图说明了在不同信号强度下通信延迟与计算延迟之间的权衡。最佳草稿长度 K^* 从 2（弱信号）移动到 6（强信号）。

Algorithm 1: FlexSpec of ine draft model training

Input : Base target model $\mathcal{M}_{\text{base}}$, dataset $\mathcal{D}_{\text{pretrain}}$

Output: Static draft model $\mathcal{M}_d = \{\mathcal{B}_{\text{shared}}, \mathcal{H}_{\text{small}}\}$

```
// Step 1: initialization and structural freezing
Initialize lightweight head  $\mathcal{H}_{\text{small}}$  randomly;
// Copy anchor block
 $\mathcal{B}_{\text{shared}} \leftarrow \mathcal{M}_{\text{base}}.\text{block}[-1]$ ;
Freeze parameters of  $\mathcal{B}_{\text{shared}}$  and  $\mathcal{M}_{\text{base}}$ ;

// Step 2: alignment training loop
for each batch  $(x, y)$  in  $\mathcal{D}_{\text{pretrain}}$  do
    // Teacher forward pass (base model)
     $h_t, z_t \leftarrow \mathcal{M}_{\text{base}}(x)$ ;
    // Student forward pass (draft model)
     $h_d, z_d \leftarrow \mathcal{H}_{\text{small}}(\mathcal{B}_{\text{shared}}(x))$ ;
    // Multi-objective loss computation
    ...;
     $\mathcal{L}_{\text{total}} \leftarrow \lambda_1 \mathcal{L}_{\text{feat}} + \lambda_2 \mathcal{L}_{\text{KD}}$ ;
    Update  $\mathcal{H}_{\text{small}}$  via optimizer( $\nabla \mathcal{L}_{\text{total}}$ );
return  $\mathcal{M}_d$ 
```

B. 考虑信道的自适应推测

在移动计算环境中，端到端延迟严格受信道动态的影响。固定的草稿长度 K 无法适应传输开销与推测收益之间的权衡。为了解决这一问题，我们将最优 K^* 选择形式化为吞吐量最大化问题，明确考虑传播延迟和可变的云端验证成本。

1) 延迟建模优化：单步投机解码的总时间消耗，步幅为 Δ ，记作 $T_{\text{step}}(K, R_n)$ 在公式 (1) 中)，由四个部分组成：边缘计算、上行传输、云端验证以及下行反馈。与假设云端验证成本恒定的先前工作不同，我们将云端延迟建模为 K 的仿射函数，该函数反映了加载 K 个新生成的标记及其相关的键值 (KV) 缓存所带来的内存带宽开销。因此，优化后的延迟模型表示为

$$\hat{T}_{\text{step}}(K, R_n) = \hat{T}_{\text{edge}}(K) + \hat{T}_{\text{up}}(K, R_n) + \hat{T}_{\text{cloud}}(K) + T_{\text{down}}. \quad (7)$$

上行传输延迟明确考虑了传播延迟 T_{prop} （等于往返时间的一半）以及由通信开销 $B_{\text{up}}(K)$

$\triangleq K \cdot b + O_{\text{header}}$ 和瞬时通信速率 R_n 决定的数据传输延迟，即，

$$\hat{T}_{\text{up}}(K, R_n) = T_{\text{prop}} + \frac{K \cdot b + O_{\text{header}}}{R_n}, \quad (8)$$

其中每个标记被编码为 b 位的整数索引， O_{header} 表示与协议相关的传输开销（例如，数据包头）。此外，云端验证延迟被建模为基础处理成本加上每个标记的边际计算成本，

$$\hat{T}_{\text{cloud}}(K) = T_{\text{base}} + K \cdot \delta_{\text{cloud}}, \quad (9)$$

其中 δ_{cloud} 表示验证一个额外令牌所产生的额外计算时间。

为了在资源受限的边缘设备上实现高效的在线优化，我们进一步将延迟项分为固定开销和边际成本。此外，我们采用线性逼近来处理边缘计算延迟，

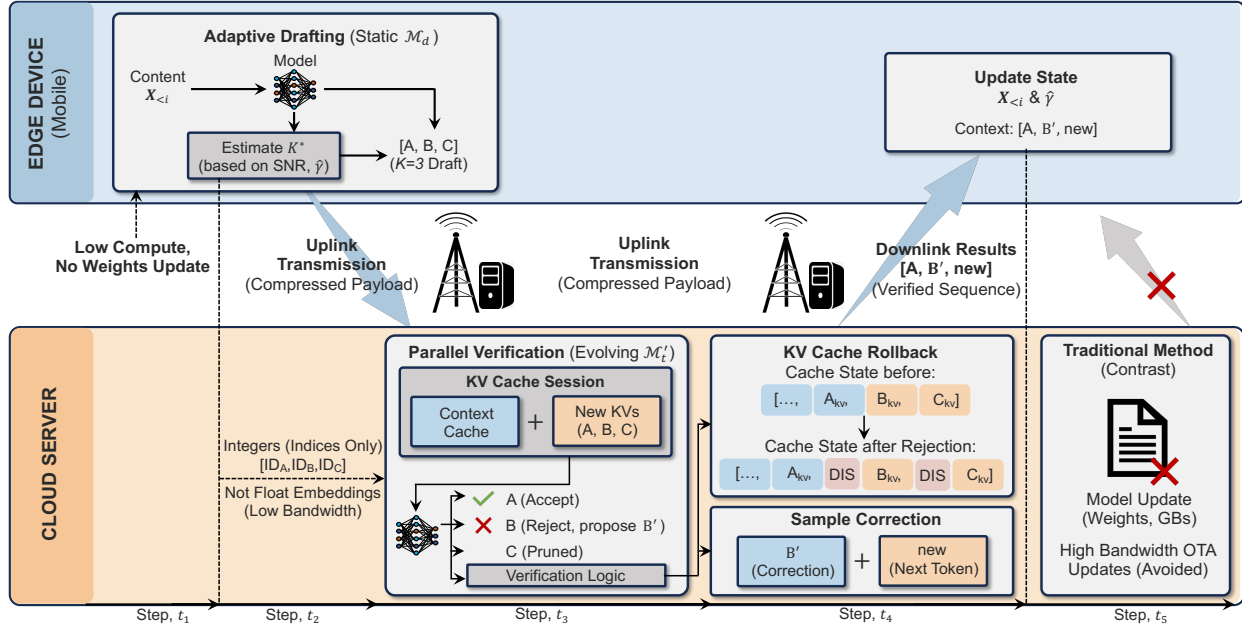


图 3. FlexSpec 的无线协作推理流程：该过程包括边缘的自适应起草（顶部）、压缩上行传输、云端的 KV 缓存回滚并验证（底部），以及经验证结果的下行传输。

$\hat{T}_{\text{edge}}(K) \approx \alpha_{\text{edge}}K + \beta$ ，这在实际中被广泛使用。代入上述表达式得到

$$\hat{T}_{\text{step}}(K, R_n) = T_{\text{xed}} + K \cdot \underbrace{\left(\alpha_{\text{edge}} + \frac{b}{R_n} + \delta_{\text{cloud}} \right)}_{T_{\text{marginal}}(n)}, \quad (10)$$

其中 $T_{\text{xed}} = T_{\text{prop}} + T_{\text{base}} + T_{\text{down}} + \frac{O_{\text{header}}}{R_n} + \beta$ ，并且 $T_{\text{marginal}}(n)$ 捕获在时变信道条件下每个额外推测标记产生的总边际延迟。

2) 吞吐量最优策略：FlexSpec 的目标是最大化公式 (2) 中的 ETGR。基于公式 (10) 中优化后的时延模型，我们根据瞬时信道条件得到公式 (2) 的可处理近似，并通过求解计算投机步长

$$K_n^* = \arg \max_{K \in [1, K_{\text{max}}]} \frac{\mathbb{E}[\tau | K]}{T_{\text{xed}} + K \cdot T_{\text{marginal}}(n)}, \quad (11)$$

其中 K_{max} 表示最大草稿长度。(11) 捕捉了标记接受增益与端到端延迟成本之间的基本权衡。

图 2 可视化了所得的优化景观。如图所示，总延迟随着无线链路质量显著变化。在信号较弱的情况下，当通信成本占主导地位时，(11) 式中的分母会随 K 快速增长，从而使最优解 K_n^* 向较小的值移动。相反，在良好的信道条件下，较大的 K 是有利的，因为传输开销可以分摊到更多的推测性标记上。

为了实现实用的在线适应，我们使用几何衰减模型或指数移动平均（EMA）来近似期望接收率 $\mathbb{E}[\tau | K]$ ，从而得到 $\mathbb{E}[\tau | K] \approx \hat{\gamma} \cdot K$ 对于适中的 K 值。在此情况下

在近似下，(11) 中的目标具有明确的物理解释。具体来说，较大的传播延迟 T_{prop} （即较大的 T_{xed} ）会促使采用更大的推测步长以分摊往返开销，而降低通信速率 R_n 会增加 $T_{\text{marginal}}(n)$ ，从而迫使 K_n^* 变小以避免拥塞。

C. FlexSpec 协议与推理流程

边缘与云之间的运行时交互被设计为相对于草稿模型版本无状态，同时相对于键值（KV）缓存保持有状态。完整的推理过程在算法 2 中进行了总结，并在图 3 中进行了说明。为了避免在云端对整个前缀历史重新计算注意力，FlexSpec 为每个用户维护一个持久的 KV 缓存会话。如图 3 的步骤 t_3 和 t_4 所示，当边缘传输 K 推测草稿标记时，云端仅为这些新接收到的标记计算相应的 KV 对。

如果在索引 $j < K$ 处发生拒绝，云端会执行 KV 缓存回滚，丢弃从索引 j 开始的所有无效 KV 对，然后再处理下一个请求。该机制确保云端的计算成本随着验证长度而变化，而不是整个上下文长度，从而保持推测解码的效率提升。

随着云端目标模型 $\mathcal{M}'_t$ 的不断发展，分布差异 $D_{\text{KL}}(P_d \| P'_t)$ 可能会随着时间增加。FlexSpec 通过轻量级的回退机制优雅地处理这种分布变化，其理念类似于在 SpecInfer [39] 中采用的方法，从而在不需要草案模型同步的情况下保持稳健的推理性能。

算法 2: FlexSpec 协同推理

e

Input : Static edge model $\mathcal{M}_d$, evolving cloud model $\mathcal{M}'_t$, context $\mathbf{x}_{<i}$, decay rate μ

Output: Generated token sequence

Initialize EMA acceptance rate $\hat{\gamma} \leftarrow 0.8$;

while EOS not generated **do**

```

    // Step 1: edge-side adaptive drafting
    Measure channel conditions;
    // Compute marginal latency and optimal speculative stride
     $T_{\text{marginal}}(n) \leftarrow \alpha_{\text{edge}} + b/R_n + \delta_{\text{cloud}}$ ;
     $K_n^* \leftarrow \arg \max_K T_{\text{xed}} + K \cdot T_{\text{marginal}}(n)$ ;
     $\mathbf{x}_{\text{draft}} \leftarrow ()$ ;
    for  $k = 1, \dots, K_n^*$  do
         $\hat{x} \sim \mathcal{M}_d(\mathbf{x}_{<i} \cdot \mathbf{x}_{\text{draft}})$ ;
         $\mathbf{x}_{\text{draft}} \leftarrow \mathbf{x}_{\text{draft}} \cdot \hat{x}$ ;
    Transmit compressed( $\mathbf{x}_{\text{draft}}$ ) to cloud via uplink;
    // Step 2: cloud-side parallel verification
    Receive  $\mathbf{x}_{\text{draft}}$  and restore KV cache;
     $\mathbf{q} \leftarrow \mathcal{M}'_t(\mathbf{x}_{\text{draft}}, \text{context} = \mathbf{x}_{<i})$ ;
     $\tau \leftarrow 0$ ;
    for  $k = 1, \dots, K_n^*$  do
        if  $x_{\text{draft}}^{(k)} == \arg \max \mathbf{q}^{(k)}$  then
             $\tau \leftarrow \tau + 1$ ;
        else
            break;
    Sample correction token  $x_{\text{new}}$  from  $\mathcal{M}'_t$ ;
    Transmit  $\mathbf{x}_{\text{verified}} \leftarrow \mathbf{x}_{\text{draft}}^{1:\tau} \cdot x_{\text{new}}$  to edge;
    // Step 3: state update
     $\mathbf{x}_{<i} \leftarrow \mathbf{x}_{<i} \cdot \mathbf{x}_{\text{verified}}$ ;
     $\hat{\gamma} \leftarrow (1 - \mu)\hat{\gamma} + \mu(\tau/K_n^*)$ ;

```

五、评估

为了全面评估 FlexSpec, 我们进行了大量实验以回答几个关键研究问题:

- **RQ1 (性能与稳健性)**: 在不同的网络条件和变化的采样温度下, FlexSpec 与最先进的基线相比表现如何?
- **RQ2 (消融研究)**: 通道感知自适应推测机制有必要吗? 在不同通道质量下, 它与固定步幅策略相比如何?
- **RQ3 (硬件通用性)**: FlexSpec 能否适应不同任务复杂度下各种移动设备的异构计算能力?
- **RQ4 (模型可扩展性)**: 该架构是否可扩展到更新的 LLM 系列 (Llama-3、Mistral) 和稀疏架构 (MoE) ?
- **RQ5 (效率)**: 在内存占用、能耗分解和热效率方面有哪些具体的收益?

A. 实验装置

扩展的硬件测试平台: 我们显著扩展了硬件多样性, 以模拟真实的跨设备部署场景。云服务器: 8× NVIDIA H800 (80GB) 通过 NVLink, 8× NVIDIA A800 (80GB) (主流), 8×NVIDIA V100 (32GB)。边缘设备 (移动客户端): NVIDIA Jetson AGX Orin (64GB 内存), Snapdragon 8 Gen 3 参考设备 (16GB 内存), Apple iPhone 15 Pro Max 模拟器 (A17 Pro), Raspberry Pi 5 (8GB 内存)。

模型和数据集: 我们在分布转移下评估 FlexSpec, 使用多样化的目标模型和下游任务套件。对于目标模型, 我们以 LLaMA-2 70B 作为代表性密集基线进行详细分析, 并进一步评估在更新的架构上的可扩展性, 包括 LLaMA-3 70B 以及专家混合模型 Mixtral 8×7B。为了全面检验任务级泛化能力, 我们考虑了六个核心下游任务, 涵盖不同的推理模式和数据分布, 包括用于数学推理的 GSM8K 数据集、用于问答的 Natural Questions、用于检索增强生成的 Natural Questions、用于多轮对话的 MT-Bench、用于机器翻译的 WMT14 DE-EN, 以及用于文档摘要的 CNN/DailyMail。

基线: 我们将 FlexSpec 与涵盖各种解码范式的扩展基线集进行了比较, 从标准自回归方法到最先进的推测框架。

• **仅云端:** 标准自回归解码完全在云服务器上进行。这作为吞吐量和延迟的基准, 其中每个生成的 token 都会产生完整的网络往返时间和云计算成本。

• **标准 SD (天真型):** 一种传统的推测性解码设置, 其中通用的预训练小模型 (例如 Llama-2-7B) 作为目标模型 (例如 Llama-2-70B) 的草稿模型。关键是, 这个基线并没有使用我们的基于锚点的对齐方法, 它代表了当草稿模型未与不断变化的目标模型同步时, 由分布变化引起的性能下降。

• **PLD (n-gram):** 提示查找解码, 一种基于检索的方法, 利用字符串匹配来识别当前上下文窗口中频繁的 n-gram 模式, 以生成未来的标记。这代表了一种轻量级、无需训练且内存高效的基线方法。

• **前瞻解码 [40]:** 一种基于雅可比迭代的并行解码算法, 可以在没有单独草稿模型的情况下同时生成多个标记。它依赖于目标模型自身的能力, 通过多步验证并行地优化多个候选序列。

• **EAGLE-2 (同步) [16]:** 当前最先进的基于模型的方法, 利用逐层特征外推。我们在“理想同步”设置下对其进行评估, 假设边缘侧扩展层已完美更新以匹配云端目标版本, 并忽略更新的相关通信开销。

• **美杜莎 -1 (同步) [14]:** 一种并行解码框架

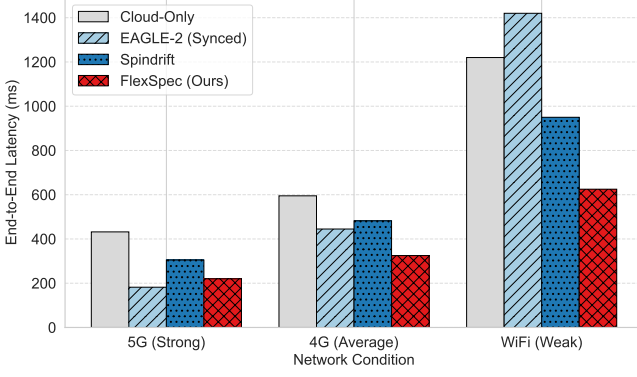


图 4. GSM8K 上的端到端延迟对比。与仅云端和最先进基线（EAGLE-2、DSSD）相比，FlexSpec 实现了显著的加速，尤其在带宽受限的环境（WiFi）下，通过有效减少通信开销。

它通过多个解码头增强目标模型以预测未来的标记。

与 EAGLE-2 类似，我们假设 Medusa 头与目标模型完美同步，以表示紧密耦合架构的理论上限。

- **DSSD [33]:** 一种专门为无线边缘云系统设计的协作推理框架。与 FlexSpec 不同，它采用固定或基于启发式的推测长度的调度策略，缺乏对变化信道条件的实时适应能力。

B. 研究问题 1：端到端延迟与网络健壮性

为了提供详细的分析，我们将评估分为两个不同的模式，即确定性生成（温度 = 0）和随机采样（温度 = 1）。我们展示了 Llama-2 70B 模型在所有六个数据集上的详细结果，以展示在各类任务上的稳健性。

1) 方案 A：确定性生成（温度 = 0）：图 4 展示了在 GSM8K 上的性能差距，表 III 使用贪婪解码呈现了详细的数值结果。此设置有利于依赖严格分布匹配的方法。

如表 III 所示，FlexSpec 在推理密集型任务（GSM8K）和抽取型任务（CNN/DM）中均提供了一致的加速效果。值得注意的是，在 GSM8K 上，由于标准 SD 在 WiFi 中因起草性能差而表现不佳（0.66 $\times$ ），FlexSpec 保持了 1.95 $\times$ 的加速，验证了我们基于锚点的对齐方法的有效性。虽然同步方法如 EAGLE-2 在 5G 中表现最佳（最高可达 2.47 $\times$ ），但在 WiFi 环境中由于传输开销而崩溃（< 0.9 $\times$ ）。通过根据信道状态调整草稿长度 K ，FlexSpec 仍然是弱网络唯一可行的加速解决方案。

2) RegimeB：随机采样（温度 = 1）：表 IV 使用 Top-p 采样（ $p = 0.9, T = 1$ ）评估性能。这强调了静态草稿模型针对变化中的云目标的对齐能力，因为草稿分布必须覆盖目标的概率质量。

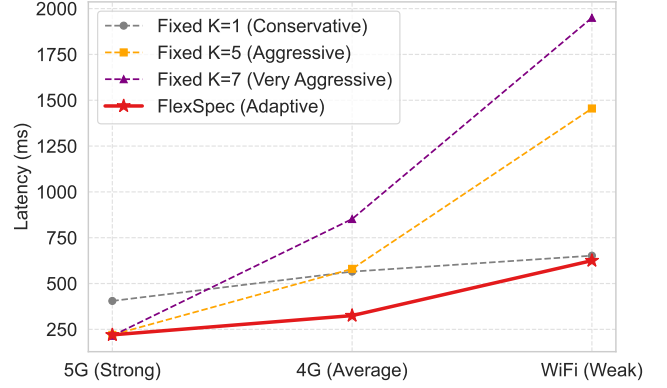


图 5. 投机跳步（ K ）对延迟的影响。固定跳步无法适应不同的信道条件（例如， $K = 5$ 会导致 WiFi 超时）。FlexSpec 的自适应机制动态选择合适的投机跳步，从而实现更优的性能。

FlexSpec 即使在随机采样下，在所有六个任务中仍保持强劲的加速（1.65 $\times$ -1.90 $\times$ ）。相比之下，像 EAGLE-2 这样的方法在温度 = 1 下会遭受严重退化（在温度 = 0 下从 2.4 $\times$ 下降到温度 = 1 下的 1.6 $\times$ ，用于 5G），因为它们的草稿生成与贪心路径紧密耦合。

如表 IV 所示，标准 SD 表现出我们动机预测的“性能崩溃”现象。尤其是在弱网络中的 GSM8K 等专门领域，标准 SD 导致显著减速（0.65 $\times$ ），因为未对齐草稿模型未能匹配目标的随机分布，导致频繁拒绝。同样，Lookahead 在此模式下几乎没有带来显著的收益（< 1.06 $\times$ ），其 n-gram 匹配能力被 random 化的令牌选择严重限制。

对于像 MT-Bench 和 CNN/DM 这样的开放式任务，FlexSpec 的基于锚点的对齐确保静态边缘草稿模型生成的标记在统计上与目标的概率分布对齐，从而避免在标准 SD 中出现的性能下降，而无需重新训练模型。

C. 研究问题 2：通道感知适应的消融研究

为了验证通道感知自适应预测机制的必要性，我们进行了一项消融研究，将动态调整替换为固定的预测步幅，同时保持基于锚点的对齐不变。实验在 GSM8K 数据集上，在三个具有代表性的网络环境中进行。

图 5 中的结果展示了所提出的自适应策略在异构网络条件下的有效性。较大的固定推测步幅（例如， $K = 5$ ）在高带宽 5G 条件下实现了低延迟（220.0 毫秒），但在弱 WiFi 场景下表现不佳（1455.0 毫秒），由于过多的传输延迟，相对于基线导致 2.3 $\times$ 的减速。相反，保守的步幅（例如， $K = 1$ ）在弱网络下仍然稳健。

表 III REGIME A (TEMPERATURE = 0): LLAMA-2 70B 在 ALL 6 DATASETS 上的端到端延迟和加速比。在各种任务的带宽受限环境中, FlexSpec 展现出优越的性能。

Dataset	Network	Cloud-Only	Lookahead	Std. SD	Medusa-1	EAGLE-2	DSSD	FlexSpec
Sync Required?		No	No	No	Yes	Yes	No	No
GSM8K (Math)	5G (Strong)	432.0ms (1.0×)	415.0ms (1.04×)	392.0ms (1.10×)	205.0ms (2.10×)	182.0ms (2.37×)	305.0ms (1.41×)	220.0ms (1.96×)
	4G (Avg)	595.0ms (1.0×)	580.0ms (1.02×)	805.0ms (0.74×)	480.0ms (1.24×)	445.0ms (1.33×)	482.0ms (1.23×)	325.0ms (1.83×)
	WiFi (Weak)	1220.0ms (1.0×)	1205.0ms (1.01×)	1850.0ms (0.66×)	1455.0ms (0.84×)	1420.0ms (0.86×)	950.0ms (1.28×)	625.0ms (1.95×)
Natural Questions (QA)	5G (Strong)	415.0ms (1.0×)	390.0ms (1.06×)	345.0ms (1.20×)	192.0ms (2.16×)	168.0ms (2.47×)	275.0ms (1.51×)	202.0ms (2.05×)
	4G (Avg)	570.0ms (1.0×)	552.0ms (1.03×)	685.0ms (0.83×)	435.0ms (1.31×)	405.0ms (1.41×)	452.0ms (1.26×)	295.0ms (1.93×)
	WiFi (Weak)	1185.0ms (1.0×)	1165.0ms (1.02×)	1680.0ms (0.71×)	1380.0ms (0.86×)	1352.0ms (0.88×)	905.0ms (1.31×)	582.0ms (2.04×)
Natural Questions (RAG)	5G (Strong)	428.0ms (1.0×)	402.0ms (1.06×)	360.0ms (1.19×)	198.0ms (2.16×)	175.0ms (2.44×)	288.0ms (1.49×)	212.0ms (2.02×)
	4G (Avg)	582.0ms (1.0×)	568.0ms (1.02×)	735.0ms (0.79×)	455.0ms (1.28×)	422.0ms (1.38×)	468.0ms (1.24×)	308.0ms (1.89×)
	WiFi (Weak)	1208.0ms (1.0×)	1190.0ms (1.02×)	1765.0ms (0.68×)	1425.0ms (0.85×)	1390.0ms (0.87×)	932.0ms (1.30×)	600.0ms (2.01×)
MT-Bench (Chat)	5G (Strong)	420.0ms (1.0×)	398.0ms (1.05×)	358.0ms (1.17×)	202.0ms (2.08×)	178.0ms (2.36×)	295.0ms (1.42×)	215.0ms (1.95×)
	4G (Avg)	578.0ms (1.0×)	562.0ms (1.03×)	742.0ms (0.78×)	462.0ms (1.25×)	430.0ms (1.34×)	475.0ms (1.22×)	312.0ms (1.85×)
	WiFi (Weak)	1192.0ms (1.0×)	1175.0ms (1.01×)	1780.0ms (0.67×)	1440.0ms (0.83×)	1405.0ms (0.85×)	945.0ms (1.26×)	615.0ms (1.94×)
WMT14 (Trans)	5G (Strong)	418.0ms (1.0×)	395.0ms (1.06×)	350.0ms (1.19×)	195.0ms (2.14×)	172.0ms (2.43×)	282.0ms (1.48×)	208.0ms (2.01×)
	4G (Avg)	575.0ms (1.0×)	560.0ms (1.03×)	720.0ms (0.80×)	445.0ms (1.29×)	415.0ms (1.38×)	460.0ms (1.25×)	302.0ms (1.90×)
	WiFi (Weak)	1188.0ms (1.0×)	1168.0ms (1.02×)	1725.0ms (0.69×)	1405.0ms (0.85×)	1372.0ms (0.87×)	918.0ms (1.29×)	592.0ms (2.00×)
CNN/DM (Summ)	5G (Strong)	425.0ms (1.0×)	400.0ms (1.06×)	355.0ms (1.20×)	198.0ms (2.15×)	175.0ms (2.43×)	285.0ms (1.49×)	210.0ms (2.02×)
	4G (Avg)	582.0ms (1.0×)	565.0ms (1.03×)	728.0ms (0.80×)	452.0ms (1.29×)	420.0ms (1.38×)	465.0ms (1.25×)	306.0ms (1.90×)
	WiFi (Weak)	1205.0ms (1.0×)	1180.0ms (1.02×)	1750.0ms (0.69×)	1420.0ms (0.85×)	1382.0ms (0.87×)	924.0ms (1.30×)	598.0ms (2.01×)

表 IV REGIME B (T=1): LLAMA-2 70B 在所有 6 个数据集上的端到端延迟和加速。FLEXSPEC 在随机抽样下保持稳健性, 而其他方法性能下降。

Dataset	Network	Cloud-Only	Lookahead	Std. SD	Medusa-1	EAGLE-2	DSSD	FlexSpec
Sync Required?		No	No	No	Yes	Yes	No	No
GSM8K (Math)	5G (Strong)	435.0ms (1.0×)	420.0ms (1.04×)	405.0ms (1.07×)	272.0ms (1.60×)	245.0ms (1.77×)	345.0ms (1.26×)	232.0ms (1.87×)
	4G (Avg)	598.0ms (1.0×)	585.0ms (1.02×)	825.0ms (0.72×)	665.0ms (0.90×)	592.0ms (1.01×)	540.0ms (1.11×)	360.0ms (1.66×)
	WiFi (Weak)	1225.0ms (1.0×)	1210.0ms (1.01×)	1880.0ms (0.65×)	1852.0ms (0.66×)	1680.0ms (0.73×)	1150.0ms (1.06×)	705.0ms (1.74×)
Natural Questions (QA)	5G (Strong)	418.0ms (1.0×)	395.0ms (1.06×)	355.0ms (1.18×)	290.0ms (1.44×)	258.0ms (1.62×)	332.0ms (1.26×)	222.0ms (1.88×)
	4G (Avg)	575.0ms (1.0×)	560.0ms (1.03×)	700.0ms (0.82×)	655.0ms (0.88×)	590.0ms (0.97×)	528.0ms (1.09×)	348.0ms (1.65×)
	WiFi (Weak)	1190.0ms (1.0×)	1172.0ms (1.02×)	1700.0ms (0.70×)	1820.0ms (0.65×)	1665.0ms (0.71×)	1110.0ms (1.07×)	690.0ms (1.72×)
Natural Questions (RAG)	5G (Strong)	430.0ms (1.0×)	408.0ms (1.05×)	370.0ms (1.16×)	285.0ms (1.51×)	255.0ms (1.69×)	338.0ms (1.27×)	228.0ms (1.89×)
	4G (Avg)	588.0ms (1.0×)	575.0ms (1.02×)	750.0ms (0.78×)	660.0ms (0.89×)	595.0ms (0.99×)	532.0ms (1.10×)	352.0ms (1.66×)
	WiFi (Weak)	1215.0ms (1.0×)	1198.0ms (1.01×)	1785.0ms (0.68×)	1840.0ms (0.66×)	1675.0ms (0.72×)	1130.0ms (1.07×)	700.0ms (1.73×)
MT-Bench (Chat)	5G (Strong)	428.0ms (1.0×)	405.0ms (1.06×)	368.0ms (1.16×)	285.0ms (1.50×)	252.0ms (1.70×)	329.0ms (1.30×)	225.0ms (1.90×)
	4G (Avg)	585.0ms (1.0×)	570.0ms (1.03×)	760.0ms (0.77×)	650.0ms (0.90×)	585.0ms (1.00×)	522.0ms (1.12×)	344.0ms (1.70×)
	WiFi (Weak)	1210.0ms (1.0×)	1195.0ms (1.01×)	1795.0ms (0.67×)	1805.0ms (0.67×)	1652.0ms (0.73×)	1100.0ms (1.10×)	685.0ms (1.76×)
WMT14 (Trans)	5G (Strong)	420.0ms (1.0×)	400.0ms (1.05×)	360.0ms (1.17×)	295.0ms (1.42×)	260.0ms (1.61×)	335.0ms (1.25×)	225.0ms (1.87×)
	4G (Avg)	578.0ms (1.0×)	565.0ms (1.02×)	735.0ms (0.79×)	675.0ms (0.85×)	605.0ms (0.95×)	530.0ms (1.09×)	350.0ms (1.65×)
	WiFi (Weak)	1195.0ms (1.0×)	1180.0ms (1.01×)	1740.0ms (0.69×)	1900.0ms (0.63×)	1690.0ms (0.71×)	1120.0ms (1.07×)	715.0ms (1.67×)
CNN/DM (Summ)	5G (Strong)	425.0ms (1.0×)	402.0ms (1.06×)	365.0ms (1.16×)	300.0ms (1.42×)	265.0ms (1.60×)	338.0ms (1.26×)	228.0ms (1.86×)
	4G (Avg)	582.0ms (1.0×)	568.0ms (1.02×)	740.0ms (0.79×)	680.0ms (0.85×)	610.0ms (0.95×)	535.0ms (1.09×)	352.0ms (1.65×)
	WiFi (Weak)	1200.0ms (1.0×)	1185.0ms (1.01×)	1765.0ms (0.68×)	1915.0ms (0.63×)	1700.0ms (0.71×)	1125.0ms (1.07×)	720.0ms (1.67×)

网络条件, 但在 5G 环境中显著低估了可用带宽 (405.0 毫秒 对比 220.0 毫秒)。相比之下, FlexSpec 会根据瞬时信道条件动态调整推测步幅。在有利的 5G 条件下, 它实现了 $K = 5$ 的低延迟 (220.0 毫秒), 同时自动降低有效步幅以接近 WiFi 场景中的 $K = 1$ 行为 (625.0 毫秒), 从而显著降低与静态推测策略相关的尾延迟风险。

D. RQ3: 异构边缘硬件适应性

我们在配置中对特定 c 边缘设备进行了 FlexSpec 的评估。为了更好地理解硬件约束如何与任务复杂性相互作用, 我们测量了三个代表性数据集的加速比: GSM8K (复杂逻辑)、MT-Bench (聊天) 和 HumanEval (代码)。

如表 V 所示, FlexSpec 的可行性取决于本地起草速度与网络传输节省之间的比率。仅依靠 CPU 的 Raspberry Pi 5, 其起草速度为

6.9 令牌 / 秒, 这比包括网络延迟在内的有效云生成速率要慢。这导致系统减慢 (0.72× - 0.85×), 确立了硬件下限: FlexSpec 需要加速器支持 (GPU/NPU)。

消费级移动设备 (iPhone 15 Pro Max 和 Snapdragon 8 Gen 3) 展示了接近工作站级 Jetson AGX Orin 的惊人加速性能。通过将庞大的 70B 目标卸载到云端, 仅在 NPU/Metal 后端运行轻量化的对齐草稿, FlexSpec 在标准智能手机上实现了互动式 LLM 体验。

在更困难的任务如 HumanEval 上, 由于较低的接受率需要更多的验证轮次, 所有设备的加速幅度略有下降 (例如, 在 Jetson 上从 2.10× 降至 1.88×)。然而, 支持 NPU 的设备仍保持超过 1.75× 的强劲加速, 确认基于锚点的对齐即使在边缘硬件上进行复杂代码生成时仍然有效。

表 V FLEXSPEC 在 4G 网络条件下异构边缘设备上的性能（相对于仅云的加速比）。

Device	Processor	Draft Latency	Draft Thruput	GSM8K (Hard)	MT-Bench (Med)	HumanEval (Hard)
Raspberry Pi 5	Cortex-A76 (CPU)	145 ms/token	6.9 tok/s	0.76× (Slowdown)	0.85× (Slowdown)	0.72× (Slowdown)
Jetson AGX Orin	Ampere GPU	8.5 ms/token	117.6 tok/s	1.96×	2.10×	1.88×
iPhone 15 Pro Max	A17 Pro (NPU)	12.0 ms/token	83.3 tok/s	1.82×	1.92×	1.75×
Snapdragon 8 Gen 3	Hexagon NPU	10.5 ms/token	95.2 tok/s	1.93×	2.05×	1.85×

表 VI 新一代模型架构上 FlexSpec 的可扩展性（数据集：MT-BENCH，网络：5G/4G）。

Target Model	Arch.	Baseline Latency	FlexSpec (5G)	FlexSpec (4G)
Llama-2-70B	Dense	420.0ms / 578.0ms	1.95×	1.85×
Llama-3-70B	Dense	395.0ms / 550.0ms	2.30×	1.92×
Mixtral 8x7B	MoE	320.0ms / 485.0ms	1.75×	1.68×

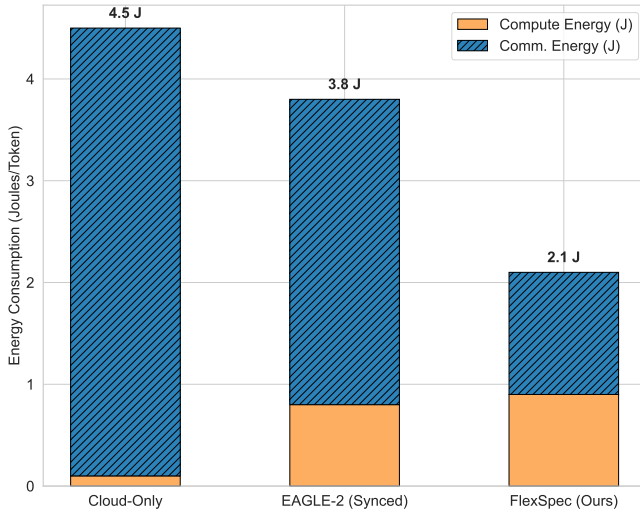


图 6. 移动设备上的能耗分解。与仅云端推理相比，FlexSpec 通过压缩令牌传输显著降低了通信能耗（蓝色斜线区域），从而实现了总体能耗降低 53%。

E. RQ4: 模型可扩展性

为了验证我们的基于锚点的对齐并非仅适用于 Llama-2 架构，我们将评估扩展到更新的 Llama-3 70B 以及稀疏专家混合模型（MoE）Mixtral 8x7B。

尽管 Llama-3 拥有更大的词汇量和不同的训练数据，FlexSpec 在 5G 条件下仍实现了 2.30× 的峰值加速（表 VI）。这表明语义锚点概念可以在密集模型中迁移。对于 Mixtral 8x7B，基线云推理速度更快，因为采用了条件计算（活跃参数 $\approx 13B$ ）。虽然这减少了推测性增益的潜在空间，但 FlexSpec 在 4G 上仍实现了 1.68× 的加速。通道感知策略会自动下调 K ，以考虑更快的云端验证，从而防止过度推测。

F. 研究问题 5: 内存与能效

最后，我们对 Jetson AGX Orin（工作站）和 Snapdragon 8 Gen 3（移动设备）的资源消耗进行了详细分析，以量化效率提升。

对于一个 70B 模型，全设备推理需要大约 42.5 GB 的显存（4-bit），在 Jetson 上可行，但对于手机则不可能。FlexSpec 通常只需要不到 ~ 3.5 GB 用于草稿模型组件，能够轻松适应现代智能手机 12-16GB 的内存限制。图 6 揭示了 FlexSpec 能源效率的来源。仅云计算的方法消耗大量能量（4.5 J/令牌），主要由于无线电尾状态（通信能量）。通过在本地起草 K 个令牌并以压缩突发方式发送，FlexSpec 显著减少了无线电活动时间（通信能量降至 1.2 J），相比标准流式传输总能量减少 53%。在设备上完全运行大模型会产生显著热量（在 Jetson 上高于 80°C ），导致热节流。FlexSpec 将主要计算任务转移到云端，维持适合手持使用的热状况（低 - 中）。

六、结论

在本文中，我们提出了 FlexSpec，一种基于投机解码的面向不断演进的边缘 - 云系统的高效通信协作推理框架。FlexSpec 的设计旨在解决现有基于 SD 的方法在频繁的云端模型更新和动态无线条件下的可扩展性限制。为此，我们首先引入了一种共享骨干架构，使得单一静态的边缘端草稿模型能够与一系列不断演进的云端目标模型兼容，从而消除了重复的边缘端重新训练和模型同步。然后，我们进一步开发了一种感知信道的自适应投机机制，在时变网络条件下动态调整投机草稿长度，以平衡端到端延迟、通信开销和设备能耗。最后，我们通过大量实验评估了 FlexSpec，结果表明 FlexSpec 一直在性能上优于传统方法。

参考文献

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F.L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat 等, “Gpt-4 技术报告,” arXiv 预印本 arXiv:2303.08774, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale 等, “Llama 2: 开放基础模型和微调聊天模型,” arXiv 预印本 arXiv:2307.09288, 2023.

- [3] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican 等, “Gemini: 一系列高性能多模态模型,” arXiv 预印本 arXiv:2312.11805, 2023.[4] Y. Li, H. Cai, R. Kong, X. Chen, J. Chen, J. Yang, H. Zhang, J. Li, J. Wu, Y. Chen 等, “迈向 AI 搜索范式,” arXiv 预印本 arXiv:2506.17188, 2025.[5] Y. Li, H. Zhang, Y. Zhang, X. Ma, W. Ye, N. Song, S. Wang, H. Xiong, D. Yin, 和 L. Chen, “M²oerank: 面向满意度的网络搜索多目标专家混合增强排序,” 见 2025 IEEE 第 41 届国际数据工程会议 (ICDE). IEEE, 2025, 页 4441–4454.
- [6] H. Xiong, J. Bian, Y. Li, X. Li, M. Du, S. Wang, D. Yin, 和 S. Helal, “当搜索引擎服务遇到大型语言模型: 愿景与挑战,” IEEE 服务计算汇刊, 2024.[7] Y. Li, Z. Lyu, Y. Zhang, H. Zhang, T. Peng, H. Xiong, S. Wang, L. Kong, G. Chen, 和 D
- [25] Z. Lyu, M. Xiao, J. Xu, M. Skoglund, 和 M. D. Renzo, “人数越多越好? 无线边缘网络中高效大规模 AI 模型推理,” IEEE 通信精选期刊, 2025.[26] E. Frantar, S. Ashkboos, T. Hoefer, 和 D. Alistarh, “GPTQ: 生成预训练变换器的精确后训练压缩,” 收录于 第十一届国际学习表征大会, 2023.[27] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, 和 S. Han, “AWQ: 面向设备的大语言模型压缩和加速的激活感知权重量化,” 收录于 机器学习与系统会议论文集, 2024.[28] M. Li, T. Cai, J. Cao, Q. Zhang, H. Cai, J. Bai, Y. Jia, M.-Y. Liu, K. Li, 和 S. Han, “Distrifusion: 高分辨率扩散模型的分布式并行推理,” 收录于 IEEE/CVF 计算机视觉与模式识别会议论文集, 2024.[29]